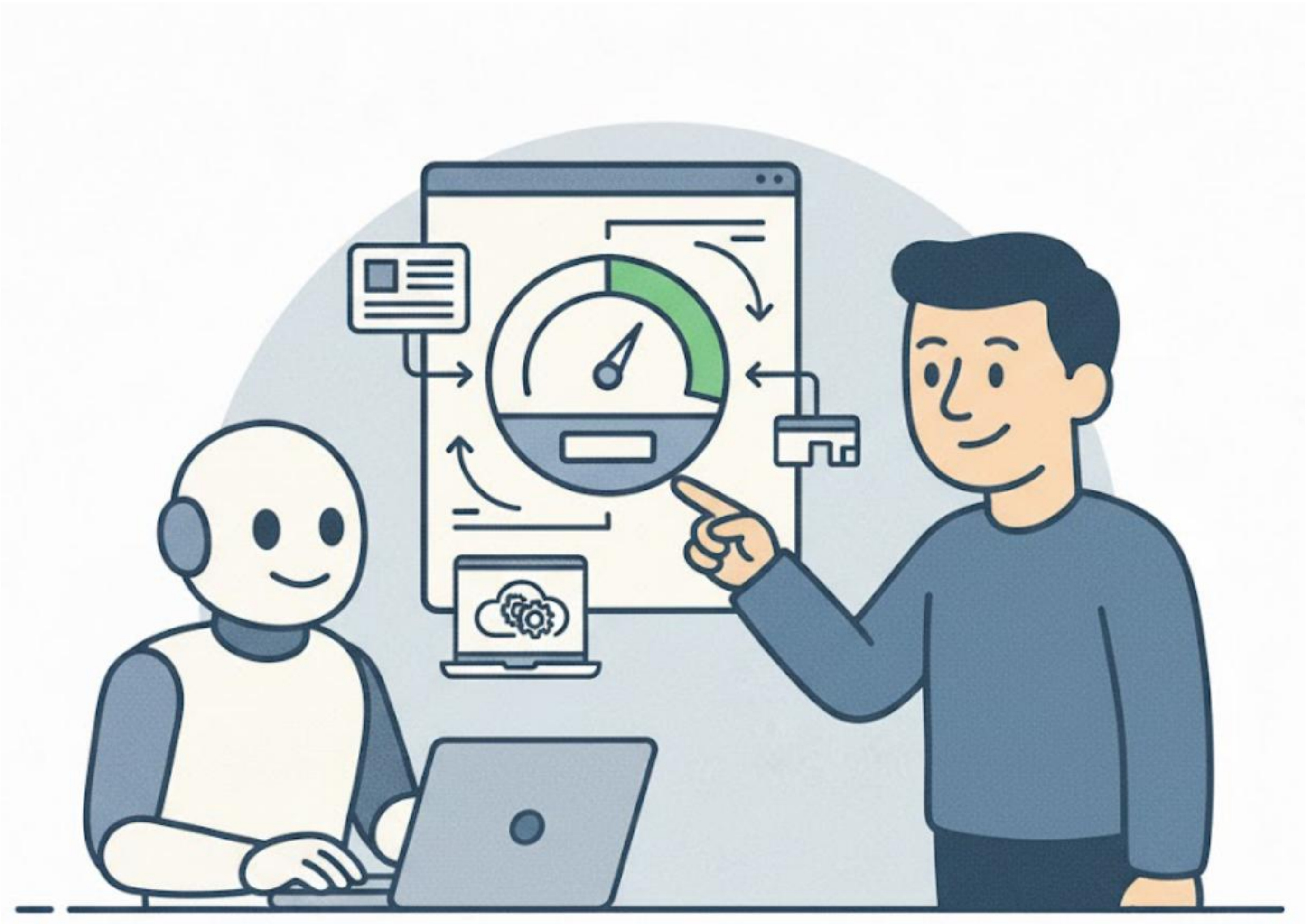
Stop Hoping, Start Evaluating!

Building AI Agents That Actually Work

Talk By: ***Tezan Sahu***



services.tezansahu@gmail.com

Little About Me...

- **Microsoft** IDC (2021 – Present)
    - **Applied Scientist II** (2025 – Present) *[M365 Copilot Extensibility]*
    - Software Engineer II (2024 – 2025) *[M365 Copilot Extensibility]*
    - Data & Applied Scientist II (2023 - 2024) *[Bing Autosuggest + Search History]*
    - Data & Applied Scientist (2021 - 2023) *[Bing Autosuggest]*
- **IIT Bombay** (2017 – 21)
    - Major in Mechanical Engineering (9.90 CPI)
    - Minor in Computer Science & Engineering
    - **Institute Rank 2** | Department Rank 1
- **Author** of "Beyond Code" – a **National Bestseller**
- **Author** of "Low-Pass Filter" – an AI Newsletter
- **7 US Patents** * | **2 Research Papers**
- AI & Data Science **Mentor**
    - Empowered **20,000+ students & professionals** in AI & Data Science

services.tezansahu@gmail.com

Agenda

Agenda

01	AI Agent Anatomy
02	5-Step Building Process
03	Introduction to Evals
04	Evaluation-Driven Development (EDD)
05	Getting Started: Tools & Resources

Anatomical Elements of AI Agents

LLM

Language model that understands queries, reasons about actions, selects tools, and generates responses.

Instruction (System Prompt)

Defines role, behavior, guidelines, and constraints. The foundation of agent personality and decision-making.

Tools & Functions

External capabilities the agent can invoke:
APIs, databases, calculators, search engines, file operations.

Orchestrator

The heart of the agent

Memory & Context

Conversation history, user preferences, past interactions, and state information for continuity.

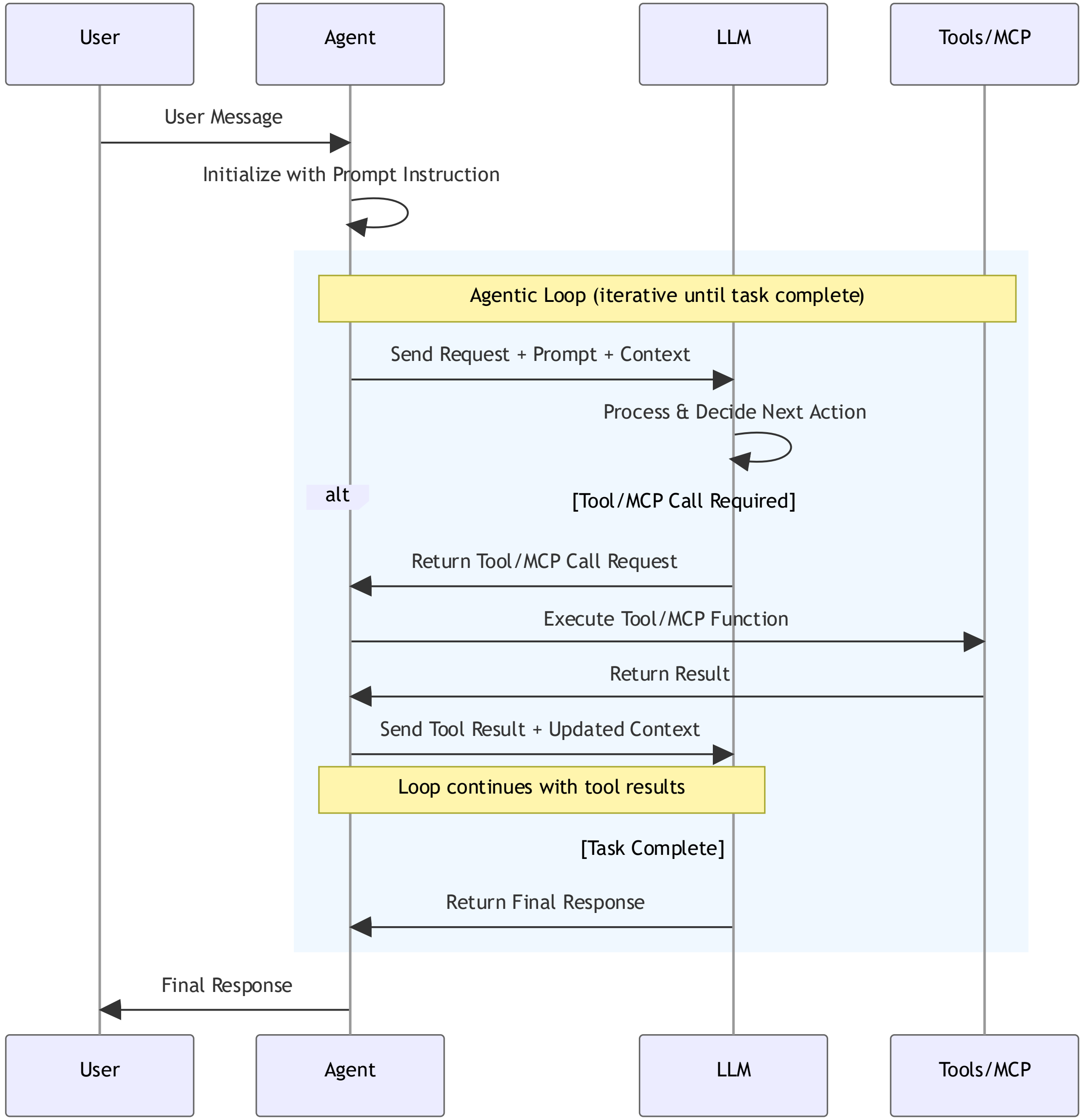
Guardrails & Safety

Validation, error handling, rate limiting, content filtering, and constraints to ensure safe operation.

Output Handler

Formats and delivers responses to users - text, structured data, or triggers for actions.

Inner Working of AI Agents



services.tezansahu@gmail.com

Agenda

Five-step building process

- **Step 1** Define the use case
- **Step 2** Choose models and tools
- **Step 3** Design the interactions
- **Step 4** Build and integrate
- **Step 5** Test and validate

services.tezansahu@gmail.com

Step 1: Define the use case

Problem Framing

- What problem are we solving?
- What will the user interactions look like?
- What is the role of AI?
- What are the current constraints and limits of AI?

Success Metrics

- What are the measurable outcomes?
- What might success look like – for users, for the business?
- What are the security and responsible AI requirements?
- Key metrics: accuracy, latency, cost, user satisfaction

Step 2: Choose models and tools

- Align with organizational standards and approved tools
- Choose between LLMs and specialized models
- Determine if agents are needed
- Determine if MCP is needed, then use as secure, auditable tool access
- Watch for over-engineering – keep it lean and purposeful

services.tezansahu@gmail.com

Step 3: Design the interactions

Craft effective prompts to guide model behavior

Inject relevant context to improve accuracy

Ground responses with trusted data sources

Apply pre- and post-guardrails for safety and reliability

services.tezansahu@gmail.com

Step 4: Build and integrate

Build the solution using selected models and tools	Integrate with approved platforms and infrastructure
Satisfy all requirements – security, responsible AI, compliance	Follow standard development practices and non-functional requirements

services.tezansahu@gmail.com

Step 5: Test, validate, and optimize

Test

- Explore edge cases
- Run bias checks
- Assess robustness and fairness

Validate

- Measure performance metrics
- Verify latency and cost expectations
- Confirm outputs meet quality and safety standards

Optimize

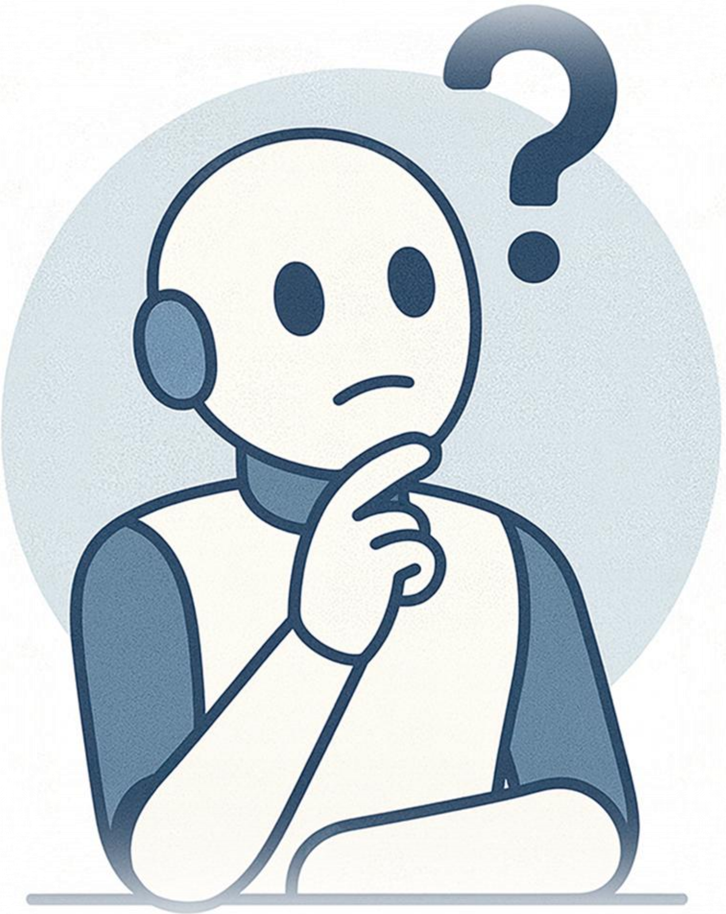
- Tune for efficiency and reliability
- Revisit model/tool choices if needed
- Avoid unnecessary complexity

Questions?

services.tezansahu@gmail.com

Agenda

services.tezansahu@gmail.com



Let’s discuss...

If an AI agent *seems* to work on your test prompt...
Is that sufficient to conclude that the agent works fine? Why?

services.tezansahu@gmail.com

Challenges with Building AI Agents

Agents behave differently each time

→ Non-determinism complicates testing and quality checks

Example: Agent gives different answers to the same question

Correctness isn’t guaranteed

→ Agents may take plausible but wrong actions

Example: Agent may extract incorrect parameters or hallucinate

The *Whac-a-Mole* Effect

→ Tuning one part of the agent system unintentionally destabilizes others

Example: Reducing hallucinations makes the agent overly conservative or breaks a previously stable tool-use flow

Traditional tests fall short

→ Dynamic, multi-run evaluations are needed

Example: Unit tests miss LLM output variability

services.tezansahu@gmail.com

Why is Evaluation of AI Agents so Critical?

💼 Business Reasons

- **Before production:** Catch issues early
- **During iteration:** Measure improvement
- **After deployment:** Monitor quality

💻 Technical Reasons

- Objective metrics > subjective feelings
- Comparison between features (which prompt/model is better?)
- Regression detection (did changes break it?)

Bottom Line: You can't improve what you don't measure

services.tezansahu@gmail.com

Evals Velocity Determines Product Velocity

The fastest-shipping AI teams share one secret…

Mike Krieger
Anthropic CPO

If there is one thing we can teach people, it's that writing evals is probably the most important thing.

Kevin Weil
OpenAI CPO

Writing evals is going to become a core skill for product managers. It is such a critical part of making a good product with AI.

Garry Tan
@garrytan

Evals are emerging as the real moat for AI startups

Hard won insights about customers and their business logic discovered by founders acting almost as ethnographers spelunking in the underserved slices of the GDP pie chart

> **Anjney Midha** @AnjneyMidha
> The best AI product leader I know makes it a habit of saying 'taste' is his differentiator publicly
>
> But behind the scenes, it's all ruthless evals

services.tezansahu@gmail.com

What Do We Evaluate?

Dimensions to Evaluate Agents On...

Task Success

Did the agent achieve the user's intended goal?

Process Quality

Was the reasoning & tool execution sequence correct? Was there any hallucination?

Reliability

Did it handle failures, retries, errors?

Safety & Compliance

Is output allowed, safe, aligned?

Efficiency

How much cost, latency, tool usage?

services.tezansahu@gmail.com

How To Evaluate These Dimensions?

Types of AI Agent Evaluations

📝 Rule-Based Evaluators

- Programmatic checks
- Good for: Format, citations, length, specific patterns
- Cheap & fast
- Example: *"Does response contain event name?"*

🤖 LLM-as-a-Judge Evaluators

- Use another LLM to judge output
- Good for: Relevance, coherence, helpfulness
- Incurs LLM costs & could be slow
- Example: *"Is this response relevant to the query?"*

💡 **Best Practice:** Use both types together – combination of evaluators for different dimensions

Offline vs Online: The Two Worlds of Evals

Offline

- **Where it happens**
  - Controlled sandbox
  - Replay datasets
- **Why do this?**
  - Ensure correctness
  - Reproducibility
  - Early debugging (pre-release)
- **What does it involve?**
  - Your own data - full visibility and debuggability
  - Scales by LLM-as-a-judge
- **Do this first!**
- Keep iterating as you learn more

Shadow

- **Where it happens?**
  - Real user inputs
  - Offline metrics
- **Why do this?**
  - Best of both worlds
- **What does this involve?**
  - Fork actual traffic into 2 streams
  - Use prod for 1 stream & updated branch for other
  - Store logs from both – for metrics
  - Show only responses from 1st stream to users

Online

- **Where it happens**
  - Live environments
  - Real user or inputs
- **Why do this?**
  - Measure real-world impact
  - Post release monitoring
- **What does it involve?**
  - A/B testing, flighting
  - Telemetry
  - Limited data (privacy)
- Usually, cannot involve LLM-as-a-judge (COGS)

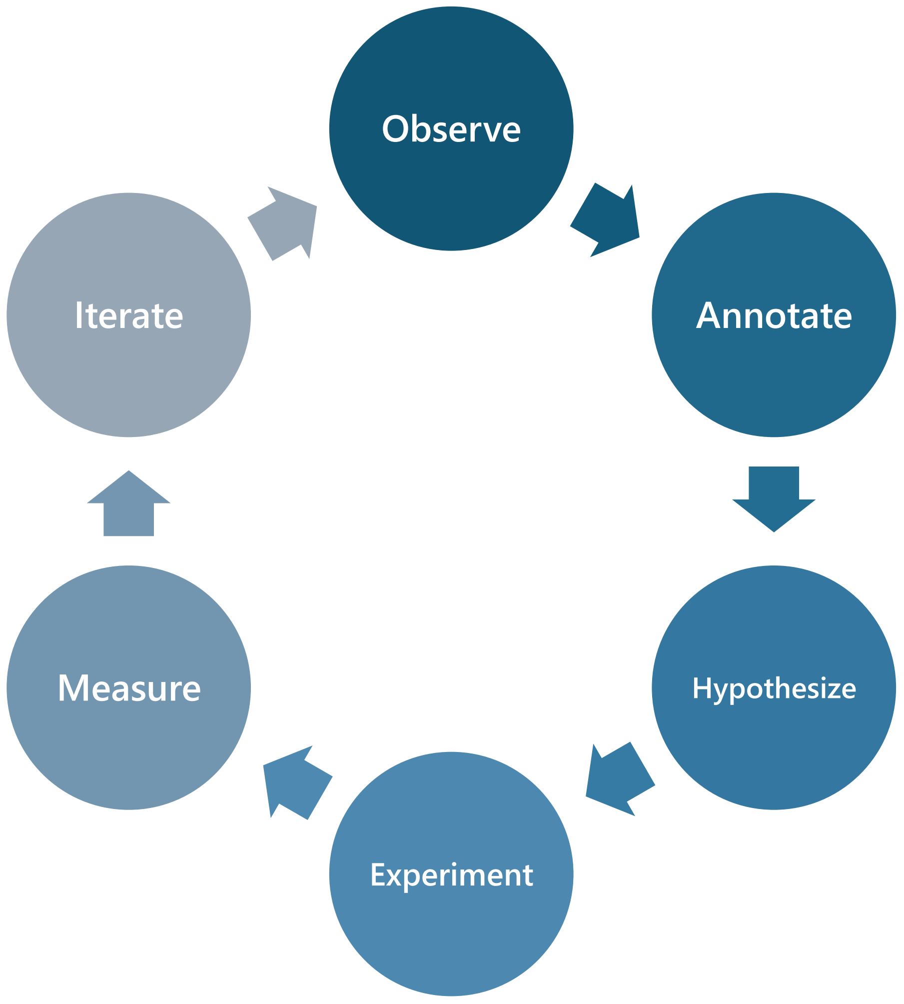
services.tezansahu@gmail.com

Agenda

services.tezansahu@gmail.com

Evaluation-Driven Development

- ☑ Build representative test datasets
- ☑ Create agent variants (different prompts, models, logic)
- ☑ Apply multiple evaluators to assess quality
- ☑ Analyze results to get actionable insights
- ☑ Iterate based on data—not intuition
- ☑ Save tricky cases and re-run them each cycle (mini “golden set”)



services.tezansahu@gmail.com

Test Dataset Design

A well-designed test dataset ensures that agent evaluation is **fair**, **reliable**, and **representative** of real-world behavior

Key Principles

Coverage of Scenarios

- Include easy, medium, and hard tasks
- Represent all relevant user intents and edge cases

Clear Success Criteria

- Define explicit constraints (budget, time, accuracy, steps)
- Provide “golden traces” or expected outcomes

Realistic Diversity

Vary input phrasing, ambiguity, context length

Capture noise, missing info, and adversarial prompts

Balance Across Dimensions

services.tezansahu@gmail.com

Evaluation Metrics: How Do We Score Agents?

- **Some Critical Metrics for State-Changing Agents:**

Metric	What it Measures
Task Success Rate	% of end-to-end tasks completed correctly
Tool Correctness	Accuracy of tool selection & parameter extraction
Relevance	On a scale of {X} to {Y}, how relevant is the response address the user's query
…	…

```
❌ "85% accuracy" → Meaningless without context
✅ "85% accuracy (blocked 12/15 unauthorized writes; failed on date formats)" → Actionable
```

- **Numbers Need Rationales**
  - Scores alone ≠ insight
  - Rationales explain **failure modes** and **edge cases.**

services.tezansahu@gmail.com

From Data to Decisions — Prioritizing Fixes

- Identify Patterns, Not Outliers
  - Look for repeated failure modes across tasks:
    - Wrong tool selection
    - Premature action (lack of planning)
    - Overthinking (too many loops)
    - Missed constraints
  - Aggregate signals across multiple metrics

- Map Failures to Fix Types
  - **Reasoning errors** → Improve system prompt, add demonstrations, introduce planning pattern
  - **Tool misuse** → Add tool descriptions, restrict tool set, improve affordances
  - **Excessive cost/latency** → Reduce reflections, cap depth, add heuristics
  - **Safety issues** → Strengthen policy prompts, add guardrails, insert filters
  - **Reliability issues** → Add retries, fallback tools, or disambiguation steps

- Prioritize High-Impact Fixes

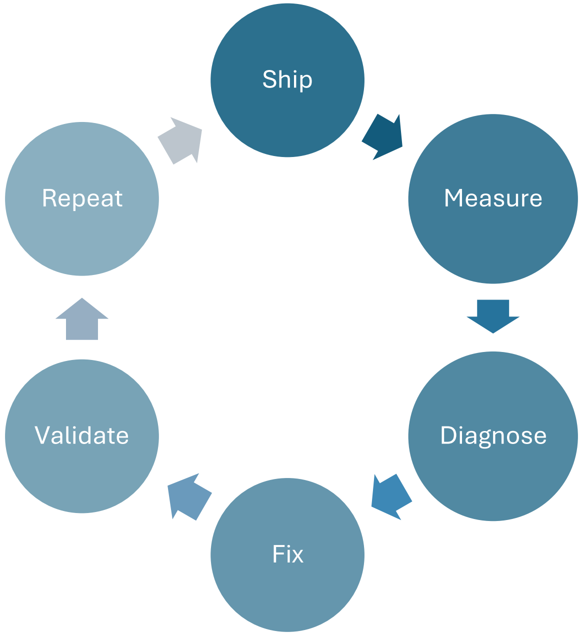
services.tezansahu@gmail.com

The Velocity Multiplier Effect

Without Strong Evals...

Cycle time: Weeks per iteration
Confidence: Low

With Strong Evals...



Cycle time: Hours to days per iteration
Confidence: High

Teams with robust eval pipelines ship 10x faster because they learn 10x faster

services.tezansahu@gmail.com

Agenda

Your Eval Toolkit: Where to Start

🎯 End-to-End Eval Frameworks

Azure AI Evaluation SDK

PromptFoo

BCG-X ArtKit

LangSmith

Braintrust

services.tezansahu@gmail.com

Your Eval Toolkit: Where to Start

⚖️ LLM-as-a-Judge Tooling

⚖️ OpenEvals

[LangChain OpenEvals]

🦾 ⚖️ AgentEvals

[LangChain AgentEvals]

AISI | AI SECURITY INSTITUTE

[Inspect AI]

[RAGAS]

services.tezansahu@gmail.com

Your Eval Toolkit: Where to Start

📊 Observability & Monitoring

Arize Phoenix

Comet Opik

LangFuse

Helicone

💡 **Pro Tip:** Start with Promptfoo or Braintrust for offline evals → Add observability later

services.tezansahu@gmail.com

Key Takeaways

Evals Velocity = Product Velocity
The fastest AI teams iterate daily because they can measure daily

Hope is not a strategy, measurement is
You can't improve what you don't measure—evals turn intuition into data

Start small, scale systematically
Begin with 10-20 representative cases → Grow as you learn failure modes

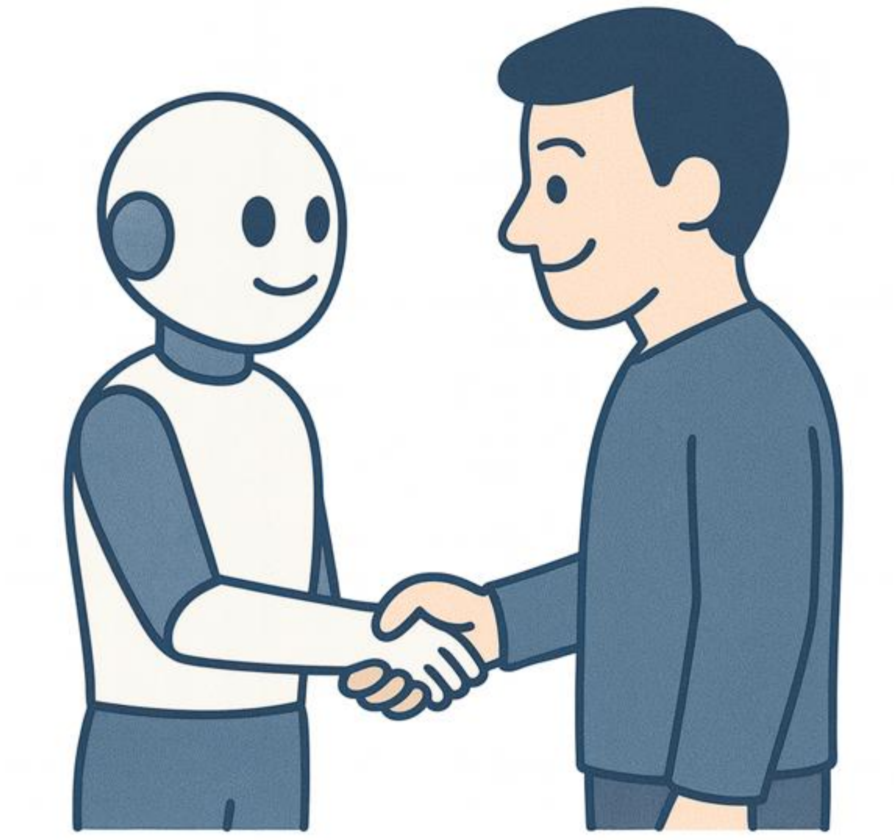
Combine evaluation approaches
Rule-based for precision + LLM-as-judge for nuance

Design for the real world
Include edge cases, ambiguity, and noise—not just happy paths

services.tezansahu@gmail.com

That’s All Folks!

For More Interesting Stuff on AI , Data Science & the Latest Technologies…



services.tezansahu@gmail.com